

Terraform

PROJECT

laC 개념 · 핵심 명령어 · 코드 리뷰 기반 프로젝트

[HISTORY]

김동진 · 이영훈 · 장규혁 · 정성현

Contents

01

Terraform

Infrastructure as Code 개념과 등장 배경 · Provider / Resource / State 등 핵심 구성요소 · 워크플로우 · 주요 명령어

02

CI / CD

HCP를 통한 Git 연동

03

프로젝트 코드 리뷰

본문 코드 구조 분석

04

인프라 구축

본 프로젝트 실습 진행

Chapter 01

IaC와 테라폼에 대해 알아보자

01

Terraform

Terraform의 기본 개념, 핵심 구성 요소, 워크플로우, 주요 명령어들을 살펴본다.

laC란 무엇인가

Infrastructure as Code — 코드로 인프라를 정의하고 관리하는 방식

BEFORE

수동 설정 (Click Ops)

- 콘솔에서 클릭으로 리소스 생성
- 같은 환경을 재현하기 어려움
- 변경 이력 추적 불가
- 휴먼 에러 발생 가능성 높음

AFTER (IaC)

코드로 인프라 정의

- 선언적(Declarative) 방식으로 상태 기술
- 동일 환경을 N번이고 재현 가능
- Git으로 변경 이력 관리
- 리뷰 / CI 파이프라인과 결합 가능

Keywords

Declarative

Idempotent

Version Control

Reproducible

Automation

Terraform 특징 및 구성요소

Provider · Resource · State · Module · Variable

01

HCL(HashiCorp Configuration Language)

테라폼 구성 파일을 작성하는 데 사용되는 언어

사람이 읽기 쉬우면서도 기계가 해석하기 용이하도록 설계되었다.

02

State (.tfstate)

Terraform이 실제 인프라와 코드의 현재 상태를 매핑해 저장하는 파일. 분실 시 매우 위험.

03

Execution Plan

terraform plan 명령어를 통해 현재 인프라 상태와 구성 파일의 목표 상태를 비교하여 어떤 리소스를 생성, 수정, 또는 삭제할지 미리 확인한다.

04

Resource

실제로 만들어지는 인프라 단위. EC2, VPC, S3 같은 객체를 코드로 선언한다.

05

Module

여러 리소스를 묶어 재사용 가능한 단위로 만든 것. 폴더 구조로 관리한다.

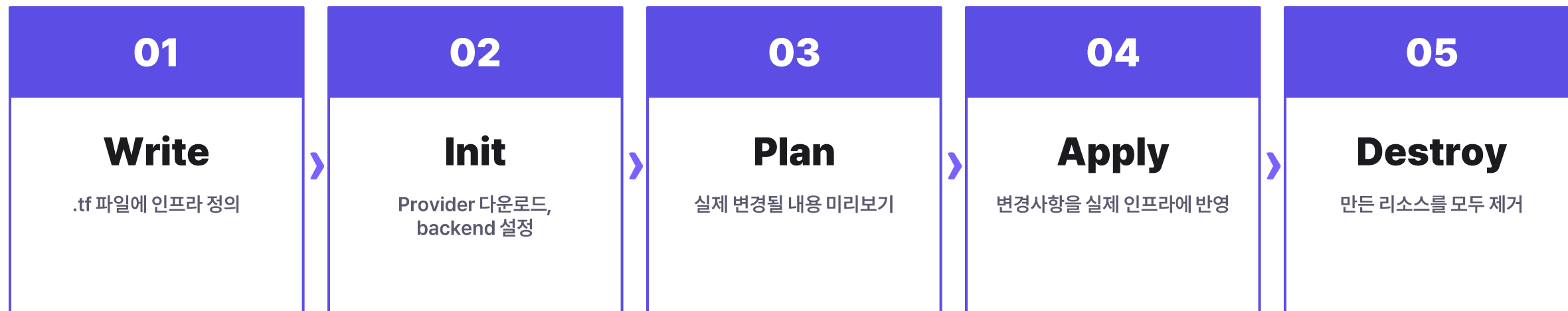
06

Backend

State 파일을 어디에 저장할지 정의 (local, S3, Terraform Cloud 등). 협업 시 원격 backend 필수.

Terraform 워크플로우

Write → Init → Plan → Apply → Destroy



💡 TIP

apply 전에는 반드시 plan으로 변경 내역을 확인할 것.
안정성을 위해 plan을 통해 변경 내역을 먼저 확인하는 것이 권장된다.

주요 명령어 — 핵심 5종 · 기타 4종

가장 자주 쓰는 명령과 옵션 정리

명령어	설명
<code>terraform init</code>	프로젝트 초기화 / provider 다운로드
<code>terraform validate</code>	문법 / 설정 검증
<code>terraform plan</code>	변경 예정 사항 미리보기
<code>terraform apply</code>	실제 인프라에 변경 적용
<code>terraform destroy</code>	관리 중인 모든 리소스 제거
<code>terraform fmt</code>	코드 포매팅 (들여쓰기 정리)
<code>terraform show</code>	인프라의 상세 정보 확인
<code>terraform output</code>	output 블록의 값 출력
<code>terraform import</code>	기존 인프라 리소스를 테라폼 으로 가져와 관리 대상으로 편입

MOST USED

1. 작업 디렉토리 초기화

```
$ terraform init
```

2. `tf.file` 문법 검증

```
$ terraform init
```

3. 변경사항 미리보기

```
$ terraform plan
```

4. 변경사항 적용

```
$ terraform apply
```

5. 리소스 제거

```
$ terraform destroy
```

명령어 상세

init · plan · apply 더 깊이 보기

terraform init

프로젝트를 처음 사용하거나 backend / provider 설정이 바뀌었을 때 실행. .terraform/ 디렉토리
와 lock 파일이 생성된다.

주요 옵션

- -upgrade : provider 버전 업그레이드
- -reconfigure : backend 재설정

terraform plan

코드와 실제 인프라의 차이를 계산해 무엇이 추가/변경/삭제될지 출력. 실제 적용은 하지 않는다.

주요 옵션

- -out=plan.tfplan : 결과를 파일로 저장
- -var-file=prod.tfvars : 변수 파일 지정

terraform apply

plan에서 본 변경사항을 실제로 적용. yes 입력 또는 -auto-approve로 진행된다.

주요 옵션

- -auto-approve : 확인 단계 생략
- plan.tfplan : plan 결과 파일 적용

Chapter 02

CI / CD 란

02

CI / CD

CI / CD의 개념과 HCP

CI / CD

CI / CD 개념

CI

지속적 통합(Continuous Integration)

코드 변경 사항을 지속적으로 통합하고 검증하는 과정

CD

지속적 배포(Continuous Deployment/Delivery)

코드 변경 사항의 통합, 테스트, 배포를 나타내는 프로세스로 두 가지 부분으로 구성된다.

지속적 배포에는 자동 프로덕션 배포 기능이 없는 반면 지속적 배포는 업데이트를 프로덕션 환경에 자동으로 릴리스한다.



CI / CD

CI / CD 효과 및 종류

CI/CD 효과

개발 생산성 향상: 수동으로 코드를 통합하고 서버에 배포하는 반복적인 작업이 사라진다.

빠른 문제 해결: 문제가 발생했을 때 어떤 코드에서 에러가 났는지 빠르게 파악하고 대응할 수 있다.

높은 신뢰성: 자동화된 테스트를 거쳐 휴먼 에러를 줄이고 안정적인 소프트웨어 버전을 유지할 수 있다.

주요 CI/CD 도구

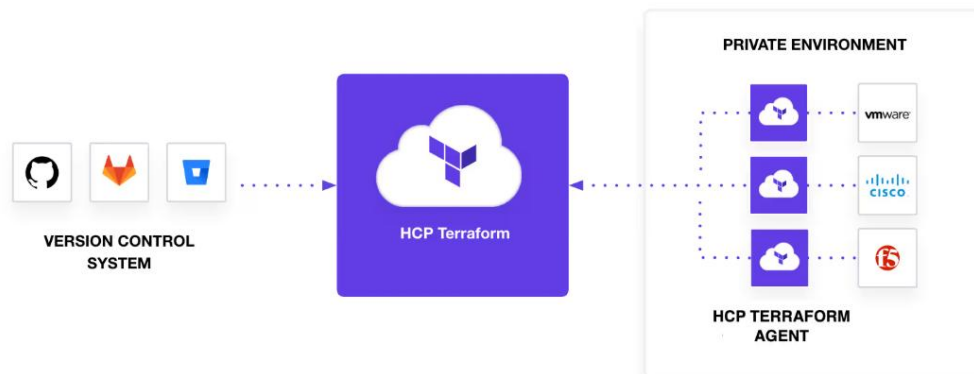
- Github Actions
- GitLab CI/CD
- Jenkins
- CircleCI
- ArgoCD

HCP Terraform 기반 GitOps CD

본 프로젝트에서는 HCP Terraform의 VCS(Version Control System) 연동 기능을 활용해 인프라의 선언적 관리와 자동 배포(GitOps) 체계 구축한다.

도입 효과

- **배포 자동화:** 인프라 자동 업데이트되어 운영 효율성 증대
- **인프라 가시성 및 협업:** 현재 인프라 상태를 확인하고 변경이력 추적 가능
- **안전성:** State 파일 유실 방지 및 충돌 방지



Chapter 03

Terraform CODE review

03

프로젝트 코드 리뷰

본문 코드 구조 분석

프로젝트 코드 리뷰

프로젝트 디렉토리 구조와 파일 역할

DIRECTORY STRUCTURE

```
project/
├── provideir.tf      # terrform & provider
├── vpc.tf            # vpc
├── subnet.tf         # subnet
├── igw.tf            # Internet GateWay
├── ngw.tf            # NAT Gateway & EIP
├── rt.tf # 실제 변수 값 # Routing Table & Routing
├── security_group.tf # Security Group
├── key_pair.tf       # Key Pair
├── ec2.tf            # Bastion Instance
├── rds.tf            # Subnet Group & Prameter Group & RDS
├── s3.tf             # S3 Bucket
├── acm.tf            # ACM
├── route53.tf        # Route53 Zone
├── alb.tf            # Target Group & Listener & ALB
└── auto_scaling.tf   # Launch Template & Auto Scaling
```

프로젝트 코드 리뷰

provider.tf 구조

provider.tf

```
1 terraform {
2   required_version = ">= 1.0.0, <2.0.0"
3
4   required_providers {
5     aws = {
6       source = "hashicorp/aws"
7       version = "~> 5.0"
8     }
9   }
10 }
11
12 provider "aws" {
13   region = "ap-northeast-2" #Asia Pacific (seoul) region
14 }
```

Terraform 환경 및 AWS Provider 설정

L 1-10

전체 프로젝트의 실행 환경 정의, 테라폼 버전 정의
AWS 프로바이더의 소스와 버전 명시

L 12-14

실제 리소스가 생성 될 클라우드 플랫폼 지정
배포 대상이 될 Region 설정

프로젝트 코드 리뷰

vpc.tf 구조

vpc.tf

```
1  ✓ resource "aws_vpc" "terraform-vpc" {  
2      |     cidr_block          = "10.250.0.0/16"  
3      |     enable_dns_support   = true  
4      |     enable_dns_hostnames = true  
5  ✓   tags = {  
6      |       "Name" = "terraform-vpc"  
7      |     }  
8  }
```

VPC 가상 네트워크 생성

L 1-8

CIDR 대역 할당
DNS 기능 활성화: 자원이 도메인 주소를 통해 통신
식별을 위해 태그 지정

프로젝트 코드 리뷰

subnet.tf 구조

L 1-3

가용영역에 분산된 퍼블릭 네트워크 생성
외부 통신을 위해 공인 IP 자동 할당 설정
전용 태그를 통해 클러스터 서비스 환경 준비

subnet.tf

```
1  # Public Subnet
2  resource "aws_subnet" "terraform-pub-subnet-2a" {
3      vpc_id            = aws_vpc.terraform-vpc.id
4      cidr_block        = "10.250.1.0/24"
5      availability_zone  = "ap-northeast-2a"
6      map_public_ip_on_launch = "true"
7      tags = {
8          "Name"                = "terraform-pub-subnet-2a"
9          "kubernetes.io/role/elb" = "1"
10         "kubernetes.io/cluster/terraform-eks-cluster" = "shared"
11     }
12 }
13
14 resource "aws_subnet" "terraform-pub-subnet-2c" {
15     vpc_id            = aws_vpc.terraform-vpc.id
16     cidr_block        = "10.250.2.0/24"
17     availability_zone  = "ap-northeast-2c"
18     map_public_ip_on_launch = "true"
19     tags = {
20         "Name"                = "terraform-pub-subnet-2c"
21         "kubernetes.io/role/elb" = "1"
22         "kubernetes.io/cluster/terraform-eks-cluster" = "shared"
23     }
24 }
```


프로젝트 코드 리뷰

subnet.tf 구조

L 27-47

외부 직접 접근이 차단된 독립적인 내부 네트워크
내부 태그 설정을 통한 서비스 간 통신 지원

```
subnet.tf
26 # Private Subnet
27 resource "aws_subnet" "terraform-pri-subnet-2a" {
28     vpc_id          = aws_vpc.terraform-vpc.id
29     cidr_block      = "10.250.11.0/24"
30     availability_zone = "ap-northeast-2a"
31     tags = {
32         "Name" = "terraform-pri-subnet-2a"
33         "kubernetes.io/role/internal-elb" = "1"
34         "kubernetes.io/cluster/terraform-eks-cluster" = "shared"
35     }
36 }
37
38 resource "aws_subnet" "terraform-pri-subnet-2c" {
39     vpc_id          = aws_vpc.terraform-vpc.id
40     cidr_block      = "10.250.12.0/24"
41     availability_zone = "ap-northeast-2c"
42     tags = {
43         "Name" = "terraform-pri-subnet-2c"
44         "kubernetes.io/role/internal-elb" = "1"
45         "kubernetes.io/cluster/terraform-eks-cluster" = "shared"
46     }
47 }
```

프로젝트 코드 리뷰

igw.tf 구조

igw.tf

```
1  resource "aws_internet_gateway" "terraform-igw" {
2    vpc_id = aws_vpc.terraform-vpc.id
3    tags = {
4      "Name" = "terraform-igw"
5    }
6  }
```

L 1-6

VPC와 인터넷 간 양방향 통신 관문 구축
퍼블릭 자원의 인터넷 접속 권한 활성화

프로젝트 코드 리뷰

ngw.tf 구조

ngw.tf

```
1  # 탄력적 IP
2  resource "aws_eip" "terraform-eip" {
3    | domain = "vpc"
4  }
5
6  # NAT 게이트웨이
7  resource "aws_nat_gateway" "terraform-ngw" {
8    | allocation_id = aws_eip.terraform-eip.id
9    | subnet_id     = aws_subnet.terraform-pub-subnet-2a.id
10   | tags = {
11     |   "Name" = "terraform-ngw"
12   | }
13 }
```

EIP 및 NAT 게이트웨이 구성

L 2-4

NAT 게이트웨이용 고정 공인 IP 확보
재시작 시에도 변하지 않는 고정 주소 할당

L 7-14

프라이빗 자원의 일방향 인터넷 접속 지원
내부 보안을 유지하며 외부 패치, 업데이트 허용

프로젝트 코드 리뷰

rt.tf 구조

rt.tf

```
1 # 라우팅 테이블
2 resource "aws_route_table" "terraform-pub-rt" {
3   vpc_id = aws_vpc.terraform-vpc.id
4
5   tags = {
6     "Name" = "terraform-pub-rt"
7   }
8 }
9
10 resource "aws_route_table" "terraform-pri-rt" {
11   vpc_id = aws_vpc.terraform-vpc.id
12
13   tags = {
14     "Name" = "terraform-pri-rt"
15   }
16 }
```

```
19 # 라우팅
20 resource "aws_route" "terraform-pub-rt" {
21   route_table_id      = aws_route_table.terraform-pub-rt.id
22   destination_cidr_block = "0.0.0.0/0"
23   gateway_id          = aws_internet_gateway.terraform-igw.id
24 }
25
26 resource "aws_route" "terraform-pri-rt" {
27   route_table_id      = aws_route_table.terraform-pri-rt.id
28   destination_cidr_block = "0.0.0.0/0"
29   nat_gateway_id      = aws_nat_gateway.terraform-ngw.id
30 }
```

```
33 # 명시적 서브넷 연결
34 resource "aws_route_table_association" "terraform-pub-rt-associate-2a" {
35   subnet_id      = aws_subnet.terraform-pub-subnet-2a.id
36   route_table_id = aws_route_table.terraform-pub-rt.id
37 }
38
39 resource "aws_route_table_association" "terraform-pub-rt-associate-2c" {
40   subnet_id      = aws_subnet.terraform-pub-subnet-2c.id
41   route_table_id = aws_route_table.terraform-pub-rt.id
42 }
43
44 resource "aws_route_table_association" "terraform-pri-rt-associate-2a" {
45   subnet_id      = aws_subnet.terraform-pri-subnet-2a.id
46   route_table_id = aws_route_table.terraform-pri-rt.id
47 }
48
49 resource "aws_route_table_association" "terraform-pri-rt-associate-2c" {
50   subnet_id      = aws_subnet.terraform-pri-subnet-2c.id
51   route_table_id = aws_route_table.terraform-pri-rt.id
52 }
```

퍼블릭 / 프라이빗 라우팅 테이블 구성

L 2-8

퍼블릭 라우팅 테이블 - vpc 연결
리소스를 쉽게 식별할 수 있도록 태그 부여

L 10-16

프라이빗 라우팅 테이블 - vpc 연결
리소스를 쉽게 식별할 수 있도록 태그 부여

L 20-24

퍼블릭 라우팅 테이블에 라우팅 규칙 추가
모든 IP 주소로 목적지 주소 정의
목적지 주소를 인터넷 게이트웨이로 보내도록 설정

L 26-30

프라이빗 라우팅 테이블에 라우팅 규칙 추가
모든 IP주소로 목적지 주소 정의
목적지 주소를 NAT 게이트웨이로 보내도록 설정

L 34-42

퍼블릭 라우팅 테이블과 서브넷을 맵핑
가용영역 2a,2c의 퍼블릭 서브넷에 규칙을 적용
퍼블릭 서브넷에 라우팅 테이블 적용

L 44-52

프라이빗 라우팅 테이블과 서브넷을 맵핑
가용영역 2a,2c의 프라이빗 서브넷에 규칙을 적용
프라이빗 서브넷에 라우팅 테이블 적용

프로젝트 코드 리뷰

security_group.tf 구조

security_group.tf

```

1 # Bastion SG
2 resource "aws_security_group" "terraform-sg-bastion" {
3   name           = "terraform-sg-bastion"
4   description    = "for Bastion Server"
5   vpc_id         = aws_vpc.terraform-vpc.id
6   tags = {
7     "Name" = "terraform-sg-bastion"
8   }
9
10  ingress {
11    from_port = -1
12    to_port   = -1
13    protocol  = "icmp"
14    cidr_blocks = ["0.0.0.0/0"]
15    description = "for ping"
16  }
17
18  ingress {
19    from_port = 22
20    to_port   = 22
21    protocol  = "tcp"
22    cidr_blocks = ["0.0.0.0/0"]
23    description = "for SSH"
24  }
25
26  ingress {
27    from_port = 80
28    to_port   = 80
29    protocol  = "tcp"
30    cidr_blocks = ["0.0.0.0/0"]
31    description = "for HTTP"
32  }

```

```

34  ingress {
35    from_port = 443
36    to_port   = 443
37    protocol  = "tcp"
38    cidr_blocks = ["0.0.0.0/0"]
39    description = "for HTTPS"
40  }
41
42  ingress {
43    from_port = 3306
44    to_port   = 3306
45    protocol  = "tcp"
46    cidr_blocks = ["0.0.0.0/0"]
47    description = "for DB"
48  }
49
50  egress {
51    from_port = 0
52    to_port   = 0
53    protocol  = "-1"
54    cidr_blocks = ["0.0.0.0/0"]
55  }
56

```

BASTION 보안 그룹 설정

L 1-8

배스천 보안 그룹 - vpc 연결
쉽게 식별할 수 있도록 태그 부여

L 10-48

인바운드 규칙(ingress 블록)
icmp 허용
22, 80, 443, 3306 포트 허용

L 50-56

아웃바운드 규칙(egress 블록)
모든 포트 허용

프로젝트 코드 리뷰

security_group.tf 구조

security_group.tf

```

58 # ALB SG
59 resource "aws_security_group" "terraform-sg-alb" {
60   name = "for ALB"
61   vpc_id = aws_vpc.terraform-vpc.id
62
63   ingress {
64     from_port = 80
65     to_port = 80
66     protocol = "tcp"
67     cidr_blocks = ["0.0.0.0/0"]
68     description = "for HTTP"
69   }
70
71   ingress {
72     from_port = 443
73     to_port = 443
74     protocol = "tcp"
75     cidr_blocks = ["0.0.0.0/0"]
76     description = "for HTTPS"
77   }
78
79   egress {
80     from_port = 0
81     to_port = 0
82     protocol = "-1"
83     cidr_blocks = ["0.0.0.0/0"]
84   }
85 }

```

```

87 # EKS SG
88 resource "aws_security_group" "terraform-sg-eks-node-group" {
89   name = "for EKS-managed server"
90   vpc_id = aws_vpc.terraform-vpc.id
91
92   ingress {
93     from_port = -1
94     to_port = -1
95     protocol = "icmp"
96     cidr_blocks = ["0.0.0.0/0"]
97     description = "for ping"
98   }
99
100   ingress {
101     from_port = 22
102     to_port = 22
103     protocol = "tcp"
104     cidr_blocks = ["0.0.0.0/0"]
105     description = "for SSH"
106   }
107
108   ingress {
109     from_port = 80
110     to_port = 80
111     protocol = "tcp"
112     cidr_blocks = ["0.0.0.0/0"]
113     description = "for HTTP"
114   }
115
116   ingress {
117     from_port = 443
118     to_port = 443
119     protocol = "tcp"
120     cidr_blocks = ["0.0.0.0/0"]
121     description = "for HTTPS"
122   }

```

ALB, EKS 보안 그룹 설정

L 59-61

ALB 보안 그룹 이름 설정
ALB 보안 그룹 - vpc 연결

L 63-77

인바운드 규칙(ingress 블록)
80, 443 포트 허용

L 79-85

아웃바운드 규칙(egress 블록)
모든 포트 허용

L 88-90

EKS 보안 그룹 이름 설정
EKS 보안 그룹 - vpc 연결

L 92-122

인바운드 규칙(ingress 블록)
icmp 허용
22, 80 포트 허용

프로젝트 코드 리뷰

security_group.tf 구조

security_group.tf

```
124  egress {  
125    from_port = 0  
126    to_port   = 0  
127    protocol  = "-1"  
128    cidr_blocks = ["0.0.0.0/0"]  
129  }  
130 }  
131  
132 # RDS SG  
133 resource "aws_security_group" "terraform-sg-rds" {  
134   name = "for RDS"  
135   vpc_id = aws_vpc.terraform-vpc.id  
136  
137   ingress {  
138     from_port = 3306  
139     to_port   = 3306  
140     protocol  = "tcp"  
141     cidr_blocks = ["0.0.0.0/0"]  
142     description = "for DB"  
143   }  
144  
145   egress {  
146     from_port = 0  
147     to_port   = 0  
148     protocol  = "-1"  
149     cidr_blocks = ["0.0.0.0/0"]  
150   }  
151 }
```

RDS 보안 그룹 설정

L 124-130 **[EKS 보안 그룹]**
아웃 바운드 규칙(egress 블록)
모든 포트 허용

L 133-135 RDS 보안 그룹 이름 설정
RDS 보안 그룹 - vpc 연결

L 137-143 인바운드 규칙(ingress 블록)
3306 포트 허용

L 145-151 아웃바운드 규칙(egress 블록)
모든 포트 허용

프로젝트 코드 리뷰

key_pair.tf 구조

key_pair.tf

```
1  ∨ resource "tls_private_key" "history_key" {  
2    algorithm = "RSA"  
3    rsa_bits  = 2048  
4  }  
5  
6  ∨ resource "aws_key_pair" "history_aws_key" {  
7    key_name   = "history"  
8    public_key = tls_private_key.history_key.public_key_openssh  
9  }  
10  
11 ∨ resource "local_file" "history_private_key" {  
12   content  = tls_private_key.history_key.private_key_pem  
13   filename = "history.pem"  
14 }
```

SSH 키 페어 자동 생성 및 관리

L 1-4

RSA 2048비트 알고리즘을 사용하여 키 생성

L 4-8

생성된 공개키를 AWS 서버에 등록

L 10-15

생성된 개인키를 .pem 파일로 로컬에 저장

프로젝트 코드 리뷰

ec2.tf 구조

ec2.tf

```
1 resource "aws_instance" "terraform-pub-ec2-bastion-2a" {
2     ami                = "ami-056a29f2eddc40520"
3     instance_type      = "t3.micro"
4     vpc_security_group_ids = [aws_security_group.terraform-sg-bastion.id]
5     subnet_id          = aws_subnet.terraform-pub-subnet-2a.id
6     key_name            = "history"
7     associate_public_ip_address = true
8
9     root_block_device {
10         volume_size = "8"
11         volume_type = "gp2"
12         tags = {
13             "Name" = "terraform-pub-ec2-bastion-2a"
14         }
15     }
16
17     tags = {
18         "Name" = "terraform-pub-ec2-bastion-2a"
19     }
20 }
```

Bastion Host 인스턴스 구성

L 2-3

AMI ID를 통한 이미지 지정
인스턴스 타입 지정

L 4-5

보안 그룹 적용
퍼블릭 서브넷 배치

L 6-7

외부 접속용 SSH키 설정
인터넷 통신용 IP 자동 할당

L 9-11

루트 볼륨 용량 및 타입 설정

L 12-14

볼륨을 위한 태그 지정

L 17-20

인스턴스 태그 지정

프로젝트 코드 리뷰

rds.tf 구조

 rds.tf

```

1 # RDS 서브넷 그룹 생성
2 resource "aws_db_subnet_group" "terraform-rds-subnet-group" {
3     name           = "terraform-rds-subnet-group"
4     subnet_ids     = [aws_subnet.terraform-pri-subnet-2a.id,
5                       aws_subnet.terraform-pri-subnet-2c.id]
6
7     tags = {
8         Name = "terraform-rds-subnet-group"
9     }
10 }
11
12 # MariaDB Parameter Group 설정
13 resource "aws_db_parameter_group" "terraform-mariadb-parameter-group" {
14     name           = "terraform-mariadb-parameter-group"
15     family         = "mariadb10.11" # 사용 중인 MariaDB 버전에 맞게 조정
16     description    = "Custom parameter group for MariaDB"
17
18     parameter {
19         name = "max_connections"
20         value = "150"
21     }
22
23     parameter {
24         name = "time_zone"
25         value = "Asia/Seoul"
26     }
27
28     parameter {
29         name = "character_set_client"
30         value = "utf8mb4"
31     }
32
33     parameter {
34         name = "character_set_connection"
35         value = "utf8mb4"
36     }
37 }

```

```
38     parameter {
39         name = "character_set_database"
40         value = "utf8mb4"
41     }
42
43     parameter {
44         name = "character_set_filesystem"
45         value = "utf8mb4"
46     }
47
48     parameter {
49         name = "character_set_results"
50         value = "utf8mb4"
51     }
52
53     parameter {
54         name = "character_set_server"
55         value = "utf8mb4"
56     }
57
58     parameter {
59         name = "collation_connection"
60         value = "utf8mb4_unicode_ci"
61     }
62
63     parameter {
64         name = "collation_server"
65         value = "utf8mb4_unicode_ci"
66     }
```

RDS 서브넷 및 파라미터 그룹 설정

L 1-9

DB가 위치할 프라이빗 서브넷들을 그룹화

L 13-21

버전 정의 및 접속 세션 최대치 설정

L 23-26

DB 서버 시간을 한국 표준시로 고정

L 28-66

이모지 및 다국어 지원을 위한 데이터셋 단일화

프로젝트 코드 리뷰

rds.tf 구조

rds.tf

```
68   parameter {
69     name = "binlog_format"
70     value = "ROW"
71   }
72 }
73
74 # RDS 생성
75 resource "aws_db_instance" "terraform-mariadb-rds" {
76   identifier_prefix = "terraform-mariadb-rds"
77   allocated_storage = 10
78   engine            = "mariadb"
79   engine_version    = "10.11"
80   instance_class     = "db.t3.micro"
81   db_name            = "terraform_mariadb_rds" # 영문자로 시작해야 하며 영문자와 숫자, _만 가능
82   username           = "root"
83   password           = "Password1234" # 8자 이상, 영문 대소문자, 숫자 및 특수 문자를 혼합하여 사용
84   parameter_group_name = "terraform-mariadb-parameter-group"
85   skip_final_snapshot = true
86   #multi_az           = true
87   db_subnet_group_name = aws_db_subnet_group.terraform-rds-subnet-group.name
88   vpc_security_group_ids = [aws_security_group.terraform-sg-rds.id]
89
90   tags = {
91     Name = "terraform-mariadb-rds"
92   }
93 }
```

RDS 구성

L 68-72

복제 및 복구를 위한 바이너리 로그 형식 지정

L 75-80

엔진 버전, 스토리지 용량 및 인스턴스 규격 지정

L 81-83

DB 명칭과 관리자 계정 및 비밀번호 설정

L 84-85사전 정의한 파라미터 그룹 연결
종료 시 임시 백업 생략 설정**L 87-88**전용 서브넷 그룹 배치 및 RDS 전용 보안 그룹을
통한 접근 제어**L 90-93**

식별용 태그 부여

프로젝트 코드 리뷰

s3.tf 구조

s3.tf

```
1 resource "aws_s3_bucket" "history-2026-terraformproject" {  
2   |   bucket = "history-2026-terraformproject"  
3   }  
4
```

S3 버킷 생성

L 1-3

실제 AWS 클라우드에 전 세계 유일한 이름으로
등록될 S3 버킷 명칭 지정

프로젝트 코드 리뷰

acm.tf 구조

```
acm.tf
1  # 인증서 생성
2  resource "aws_acm_certificate" "cert" {
3      domain_name      = "*.history-cloud.store"
4      validation_method = "DNS"
5
6      tags = {
7          Name = "history-cloud.store"
8      }
9
10     lifecycle {
11         create_before_destroy = true
12     }
13 }
14
15 # 인증서 검증
16 resource "aws_route53_record" "route53_ssl" {
17     for_each = {
18         for dvo in aws_acm_certificate.cert.domain_validation_options : dvo.domain_name => {
19             name      = dvo.resource_record_name
20             record    = dvo.resource_record_value
21             type      = dvo.resource_record_type
22         }
23     }
24
25     allow_overwrite = true
26     name            = each.value.name
27     records         = [each.value.record]
28     ttl             = 60
29     type            = each.value.type
30     zone_id         = aws_route53_zone.route53.zone_id
31 }
```

SSL/TLS 인증서 생성 및 검증

L 2-13

도메인 보안을 위한 ACM 인증서 생성
무중단 갱신 설정

L 16-31

신청한 인증서의 소유권을 증명하기 위한 Route53
DNS 레코드 자동 등록

프로젝트 코드 리뷰

route53.tf 구조

route53.tf

```
1 resource "aws_route53_zone" "route53" {  
2     |     name = "history-cloud.store"  
3     }  
4
```

Route53 구성

L 1-3

name 옵션을 사용하여 도메인의 네임서버 역할을 하고 DNS 레코드를 관리할 Route53 호스팅 영역 생성

프로젝트 코드 리뷰

alb.tf 구조

alb.tf

```
1 ##### alb 생성
2 resource "aws_lb" "web-lb" {
3   name = "web-lb"
4   subnets = [aws_subnet.terraform-pub-subnet-2a.id,
5               | aws_subnet.terraform-pub-subnet-2c.id]
6   internal = false
7   security_groups = [aws_security_group.terraform-sg-bastion.id]
8   load_balancer_type = "application"
9   tags = {
10    | "Name" = "web-lb"
11  }
12 }
13
14 ##### Target Group
15 resource "aws_lb_target_group" "terraform-prd-tg" {
16   name = "terraform-prd-tg"
17   port = 80
18   protocol = "HTTP"
19   vpc_id = aws_vpc.terraform-vpc.id
20   health_check {
21     port = 80
22     path = "/"
23   }
24   tags = {
25     | "Name" = "terraform-prd-tg"
26   }
27 }
```

```
29 #####Listener
30 resource "aws_lb_listener" "terraform-prd-listener" {
31   load_balancer_arn = aws_lb.web-lb.arn
32   port = "80"
33   protocol = "HTTP"
34   default_action {
35     target_group_arn = aws_lb_target_group.terraform-prd-tg.arn
36     type = "forward"
37   }
38 }
```

ALB & 타겟 그룹 & 리스너 설정

L 2-12

외부 트래픽을 받아 분산해 주는 퍼블릭 어플리케이션 로드밸런서 생성

L 15-27

트래픽이 최종 도달할 대상 인스턴스 그룹 정의 및 상태 검사 조건 설정

L 30-38

특정 포트로 들어오는 요청을 감지해 지정된 타겟 그룹으로 전달하도록 연동

프로젝트 코드 리뷰

auto_scaling.tf 구조

auto_scaling.tf

```
1 # 1. Launch Configuration 대신 Launch Template 사용
2 resource "aws_launch_template" "as_template" {
3   name_prefix = "terraform-lt-backend-"
4   image_id    = "ami-056a29f2eddc40520"
5   instance_type = "t3.micro"
6   key_name     = "history"
7
8   # Launch Template에서는 security_groups 대신 vpc_security_group_ids를 사용합니다.
9   vpc_security_group_ids = [aws_security_group.terraform-sg-bastion.id]
10
11   # User Data는 base64encode 처리를 해주는 것이 안전합니다.
12   # EOF/EOT 불일치 오류와 종괄호 위치를 수정했습니다.
13   user_data = base64encode(<<-EOF
14     #!/bin/bash
15     apt update
16     apt install -y nginx
17     systemctl enable nginx
18     systemctl start nginx
19     cat > /var/www/html/index.nginx-debian.html <<'HTML'
20     <!DOCTYPE html>
21     <html lang="ko">
22     <head>
23       <meta charset="UTF-8">
24       <meta name="viewport" content="width=device-width, initial-scale=1.0">
25       <title>history-cloud.store</title>
26       <style>
27         * { margin: 0; padding: 0; box-sizing: border-box; }
28         body {
29           font-family: -apple-system, "Segoe UI", Roboto, sans-serif;
30           background: #0a0a0f;
31           color: #e4e4e7;
32           min-height: 100vh;
33           display: flex;
34           align-items: center;
35           justify-content: center;
36           overflow: hidden;
37         }
38     </head>
39     <body>
40       <div>
41         <h1>history-cloud.store</h1>
42       </div>
43     </body>
44     </html>
45     <<-EOF
46 )
```

오토스케일링 시작 및 템플릿 생성

L 3-6

생성될 EC2의 이름 접두사, OS 이미지(AMI), 서버 사양, 접속용 키 페어 지정

L 9

인스턴스에 적용할 보안 그룹 지정

L 13

서버 부팅 시 자동 실행될 초기화 스크립트 선언 및 인코딩 시작

L 14-18

웹 스크립트를 통해 OS 패키지를 업데이트하고 Nginx 웹 서버를 설치한 뒤 활성화하도록 설정

L 19-36

Nginx 웹 서버의 기본 메인 페이지에 표시될 HTML 코드와 스타일(CSS) 소스 주입

프로젝트 코드 리뷰

auto_scaling.tf 구조

auto_scaling.tf

```

116 # Launch Template의 태그 지정 방식
117 tag_specifications {
118     resource_type = "instance"
119     tags = {
120         Name = "jeff-userdata"
121     }
122 }
123
124 lifecycle {
125     create_before_destroy = true
126 }
127

```

```

129 # 2. Auto-Scaling 그룹 생성 (수정된 Launch Template 참조)
130 resource "aws_autoscaling_group" "terraform-prd-asg" {
131     name = "terraform-prd-asg"
132     vpc_zone_identifier = [aws_subnet.terraform-pub-subnet-2a.id, aws_subnet.terraform-pub-subnet-2c.id]
133     min_size = 2
134     max_size = 4
135     desired_capacity = 3
136     health_check_grace_period = 120
137     health_check_type = "ELB"
138     target_group_arns = [aws_lb_target_group.terraform-prd-tg.arn]
139
140     # 기존 launch_configuration 지우고 launch_template 추가
141     launch_template {
142         id = aws_launch_template.as_template.id
143         version = "$Latest" # 언제나 최신 버전의 템플릿 적용
144     }
145
146     lifecycle {
147         create_before_destroy = true
148     }
149 }

```

오토스케일링 시작 템플릿 태그 및 수명주기 설정

L 117-122

템플릿을 통해 생성될 EC2 인스턴스에 자동으로 부여될 이름 태그 정의

L 124-126

템플릿 변경 시 기존 자원을 삭제하기 전 새 자원을 먼저 생성하도록 설정

오토스케일링 그룹 생성

L 130-135

지정된 서브넷 구역에 자동 배치될 인스턴스의 최소·최대·평상시 수량 범위 지정

L 136-138

로드밸런서 기반 상태 검사 및 타겟 그룹 자동 등록 연동

L 141-148

기반이 될 시작 템플릿의 최신 버전을 연결하고 교체 시 무중단 설정 적용

Chapter 04

Build Infrastructure

04

인프라 구축

프로젝트 실습 진행

1. app.terraform.io에 로그인
2. Create Organization

HCP Terraform

Organizations

Terraform organizations let you manage organizations, projects, and teams.



Create your first organization

+ Create organization

HCP Terraform

NOTE

Create Organization

Type: Personal

Name: history-hcp



Personal



Best for individual use for your own personal projects that have no affiliation to a business.



Terraform organization name

- Must be unique.
- May contain valid characters including ASCII letters, numbers, dashes (-), and underscores (_).

history-hcp

HCP Terraform

NOTE**조직 생성 후 화면**

1. Workspace를 생성한다.
2. workflow는 VCW 선택
VCW(Version Control Workflow)

Create a new Workspace

HCP Terraform organizes your infrastructure resources by workspaces. A workspace contains infrastructure resources, variables, state data, and run history. [Learn more about workspaces in HCP Terraform.](#)

Choose your workflow

Version Control Workflow

Trigger runs based on changes to configuration in repositories.



Best for those who need traceability and transparency

CLI-Driven Workflow

Trigger runs in a workspace using the Terraform CLI.



Best for those comfortable with Terraform CLI

API-Driven Workflow

Trigger runs using the HCP Terraform API.



Best for those with custom integrations and pipelines

Cancel

HCP Terraform

NOTE**Connect to VCS**

Project: Default Project 연결

GitHub: 깃허브 계정 연동 진행
GitHub App을 클릭해 넘어간다.

The screenshot shows a multi-step wizard for connecting to a version control system (VCS). The progress bar at the top indicates five steps: 1. Choose Type (completed), 2. Connect to VCS (current step), 3. Choose a repository, 4. Configure settings, and 5. Variable editor. The main content area is titled 'Connect to a version control provider' and instructs the user to 'Choose the version control provider that hosts the Terraform configuration for this workspace.' Below this, the 'Project' is set to 'Default Project' with an information icon. The visibility is set to 'Public'. Under the 'GitHub' section, there is a button for 'GitHub App'. At the bottom, there is a link to 'Connect to a different VCS' and navigation buttons for 'Previous' and 'Cancel'.

Choose Type Connect to VCS Choose a repository Configure settings Variable editor

Connect to a version control provider

Choose the version control provider that hosts the Terraform configuration for this workspace.

Project: Default Project ⓘ

Public

GitHub

GitHub App

[Connect to a different VCS](#)

[< Previous](#) [Cancel](#)

HCP Terraform

NOTE

다음으로 진행 전

1. Repository 생성

HCP-Terraform-HISTORY

General

Owner *



SiGgoBuk ▾

Repository name *

HCP-Terraform-HISTORY

✔ HCP-Terraform-HISTORY is available.

Great repository names are short and memorable. How about [curly-journey?](#)

Description

HCP Terraform

13 / 350 characters

HCP Terraform

NOTE

다시 **workspace** 생성 진행

1. Add another organization
2. 방금 생성한 repository 선택
3. Install

✓

✓

3

4

5

[Choose Type](#)[Connect to VCS](#)[Choose a repository](#)[Configure settings](#)[Variable editor](#)

Choose a repository

Choose the repository that hosts your Terraform source code. We'll watch this for commits and pull requests.

Don't have a repo? Here's an [example repo](#) you can copy to get started.

Organization

SiGgoBuk

✓ SiGgoBuk

Add another organization



Install Terraform Cloud

Install on your personal account DongJin Kim

for these repositories:

☐ All repositories
This applies to all current and future repositories owned by the resource owner. Also includes public repositories (read-only).

☒ Only select repositories
Select at least one repository. Also includes public repositories (read-only).

Select repositories

Selected 1 repository.

SiGgoBuk/HCP-Terraform-HISTORY

with these permissions:

✓ Read access to code, metadata, and pull requests

✓ Read and write access to commit statuses

Install

Cancel

Next: you'll be directed to the GitHub App's site to complete setup.

HCP Terraform

NOTE

Repository 확인

이전에 안보였던 repository 확인
repository를 선택하고 넘어간다.

Repository

Choose the repository that hosts your module source code.

Select a repository

Q Search

SiGgoBuk/HCP-Terraform-HISTORY

Can't find your repository? [Enter its ID manually](#) Refresh  1 total

Configure settings
Create를 눌러 생성한다.

HCP Terraform

✓

Choose Type

✓

Connect to VCS

✓

Choose a repository

4

Configure settings

5

Variable editor

Configure Settings

Workspace Name

The name of your workspace is unique and used in tools, routing, and UI. Dashes, underscores, and alphanumeric characters are permitted. Learn more about [naming workspaces](#).

Description (Optional)

Tags

Use tags to correlate, organize, or filter your resources using single values or key/value pairs.

+ Add tag

▼ Advanced options

< Previous

Cancel

Create

NOTE

생성한 Workspace 확인

HCP Terraform

Workspaces

New

All filters

Search by workspace name

0 Need attention

0 Errored

0 Running

0 On hold

0 Completed

No filters applied

Workspace name	Repository	Health	Project	Latest change	Manage
HCP-Terraform-HISTORY No status reported	SiGgoBuk/HCP-Terraform-HISTORY	None	Default Project	a minute ago	...

HCP Terraform

NOTE**Configure variables**

1. Workspace에 들어간다
2. Configure variables 클릭

history-hcp / Workspaces / HCP-Terraform-HISTORY / Overview

HCP-Terraform-HISTORY

ID: ws-JfYPAAz56GdmjTo6 

[Add workspace description.](#)

 Unlocked  Resources 0  Tags 0  Terraform v1.15.4  Updated a minute ago

 Configuration uploaded successfully

Next step: configure variables

Configure any required variables (such as access keys or configuration values) before starting a run.

[Configure variables](#)

HCP Terraform

NOTE

Configure variables

Workspace variables 블록에서
1. Add variable을 눌러 변수 생성

Workspace variables 0

Variables defined within a workspace always overwrite variables from variable sets that have the same type and the same key. Learn more about variable set [precedence](#) ↗.

+ Add variable

 Quick setup AWS dynamic credentials

HCP Terraform

NOTE**Configure variables**

다음과 같이 변수를 생성한다.

1. AWS Access Key ID
2. AWS Secret Access Key

Add variable

Select variable category

☒ Terraform variable

These variables should match the declarations in your configuration. Click the HCL box to use interpolation or set a non-string value.

☐ Environment variable

These variables are available in the Terraform runtime environment.

Key

AWS_ACCESS_KEY_ID

Value

.....

☒ HCL ☒ Sensitive

Description (Optional)

description (optional)

Add variable

Cancel

Add variable

Select variable category

☒ Terraform variable

These variables should match the declarations in your configuration. Click the HCL box to use interpolation or set a non-string value.

☐ Environment variable

These variables are available in the Terraform runtime environment.

Key

AWS_SECRET_ACCESS_KEY

Value

.....

☐ HCL ☒ Sensitive

Description (Optional)

description (optional)

Add variable

Cancel

HCP Terraform

NOTE

Configure variables
생성된 변수 확인

Workspace variables2

Variables defined within a workspace always overwrite variables from variable sets that have the same type and the same key. Learn more about variable set [precedence](#).

Key	Value	Category	Actions
AWS_ACCESS_KEY_ID HCL Sensitive	Sensitive - write only	terraform	...
AWS_SECRET_ACCESS_KEY Sensitive	Sensitive - write only	terraform	...

+ Add variable

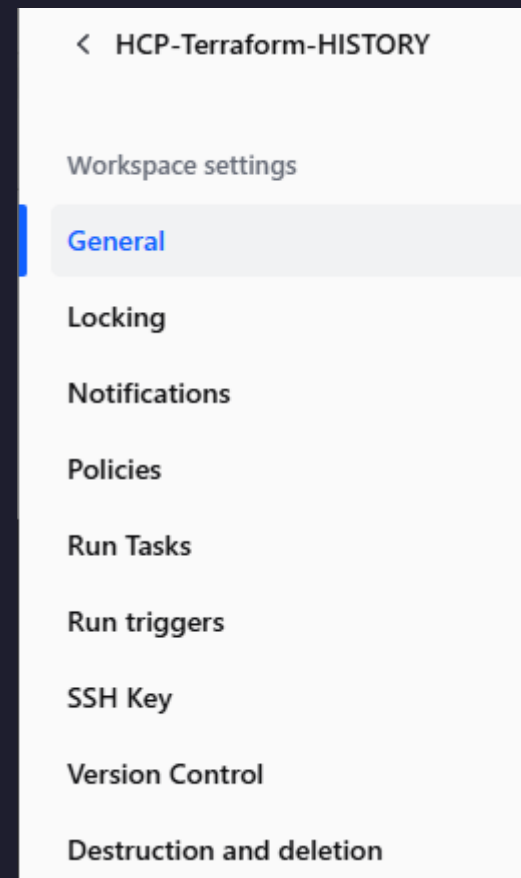
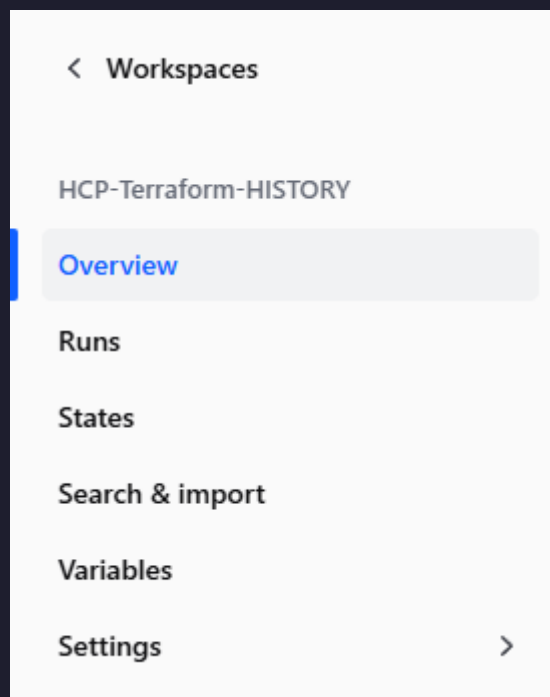
Quick setup AWS dynamic credentials

HCP Terraform

NOTE

Settings

우측 탭에서 **Settings**에 들어간다.
Settings - **General** 에서 진행



HCP Terraform

NOTE**Settings**

다음과 같이 설정을 변경한다.

1. Auto-apply
2. Remote state sharing

Auto-apply

Sets this workspace to automatically apply changes for successful runs. If disabled, runs require operator approval.

☒ Auto-apply API, UI, & VCS runs

Automatically apply changes when a Terraform plan is successful. If this workspace is linked to version control, a push to the default branch of the linked repository will trigger a plan and apply.

☒ Auto-apply run triggers

Run triggers create new plans whenever a specified source workspace completes an apply. This setting automatically applies these automatically created runs.

Remote state sharing

Choose whether this workspace should share state with the entire organization, or only with specific approved workspaces. The `terraform_remote_state` data source relies on state sharing to access workspace outputs.

☒ Share with all workspaces in this organization☐ Share with all workspaces in this project☐ Share with specific workspaces

Select workspaces



HCP Terraform

NOTE

Run

진행 전 GitHub Repository

1. tf 파일을 업로드

테스트 용도로 2개만 업로드 하였음.

 SiGgoBuk	Add files via upload	f70f5a8 · now	
 1. Terraform Provider.tf	Add files via upload	now	
 2. VPC.tf	Add files via upload	now	
 README.md	Initial commit	24 minutes ago	

Run
Workspace에서
1. New run 클릭

HCP Terraform

history-hcp / Workspaces / HCP-Terraform-HISTORY / Overview

HCP-Terraform-HISTORY

Lock

+ New run

ID: ws-JfYPAAz56GdmjTo6

[Add workspace description.](#)

Unlocked Resources 0 Tags 0 Terraform v1.15.4

Updated 2 minutes ago

✓ Configuration uploaded successfully

Next step: configure variables

Configure any required variables (such as access keys or configuration values) before starting a run.

Configure variables

SiGgoBuk/HCP-Terraform-HISTORY

Readme

Execution mode: Remote

Auto-apply API, UI, & VCS runs: On

Auto-apply run triggers: On

Auto-destroy: Off

Project: Default Project

HCP Terraform

NOTE

Run

1. Run name: apply test
2. Start

Start a new run

i

This plan will be automatically applied if checks pass, because "Auto apply" is enabled in this workspace.

Run name

apply test

Run Type

✓ Plan and apply (standard) ▼

▼ Additional planning options

Start

Cancel

HCP Terraform

NOTE

Run

apply test 적용 확인

1. run이 applied 되었는지 확인
2. AWS Console에서 확인

apply test

CURRENT

Applied

⚠ Diagnostics were found in this run

Plan & apply duration
Less than a minute

Resources changed

+1 ~0 -0

Actions

0 invoked

Run details a few seconds ago

csn862 triggered a run from UI

Plan finished 3 minutes ago

Resources: 1 to add, 0 to change, 0 to destroy | Actions: 0 to invoke

Apply finished a minute ago

Resources: 1 added, 0 changed, 0 destroyed | Actions: 0 invoked

VPC (2) 정보

Last updated
less than a minute ago



작업 ▼

VPC 생성

Q 속성 또는 태그로 VPC 찾기

< 1 > ⚙

☐	Name	VPC ID	상태
☐	terraform-vpc	vpc-09f25b7e1c683e125	Available
☐	default_vpc	vpc-09dd9fef411a76f2f	Available

HCP Terraform

NOTE

Run

1. 나머지 tf 파일 업로드
2. Github와 연동 확인

Current Run



Add files via upload

CURRENT

✓ Applied

#run-MjXF3YdnBeDpmhko

| SiGgoBuk triggered via GitHub

| Branch main

| de5cdae

a few seconds ago

HCP Terraform

NOTE**Connect**

1. Route53에서 NS 확인
2. 가비아에서 네임 서버 설정

history-cloud.store

NS

단순

-

아니요

ns-210.awsdns-26.com.
ns-948.awsdns-54.net.
ns-1368.awsdns-43.org.
ns-1973.awsdns-54.co.uk.

네임 서버

설정

네임서버 목록

- 네임서버 변경은 도메인을 연결하는 서버를 바꾸는 작업입니다.

1차 ns-210.awsdns-26.com

2차 ns-948.awsdns-54.net

3차 ns-1368.awsdns-43.org

4차 ns-1973.awsdns-54.co.uk

HCP Terraform

NOTE

Connect

1. Route53에서 A레코드 생성 (@, www)

빠른 레코드 생성

▼ 레코드 1

레코드 이름 | 정보

history-cloud.store

루트 도메인에 대한 레코드를 생성하려면 비워 둡니다.

☒ 별칭

트래픽 라우팅 대상 | 정보

별칭 호스팅 영역 ID: ZWKZPGTI48KDX

빠른 레코드 생성

▼ 레코드 1

레코드 이름 | 정보

.history-cloud.store

루트 도메인에 대한 레코드를 생성하려면 비워 둡니다.

☒ 별칭

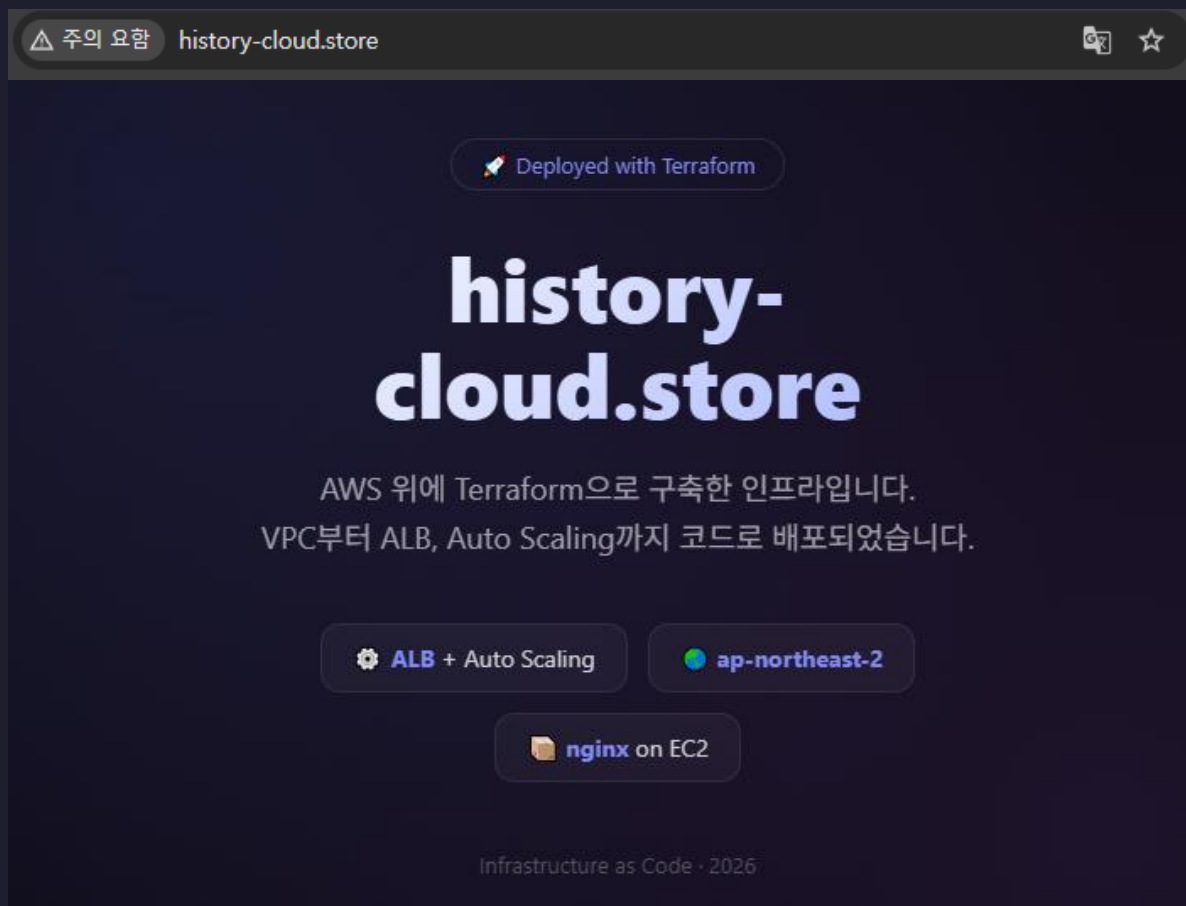
트래픽 라우팅 대상 | 정보

별칭 호스팅 영역 ID: ZWKZPGTI48KDX

HCP Terraform

NOTE

Connect
history-cloud.store 접속 확인





HISTORY